Project Proposal: EvalScope — LLM Evaluation & Observability Platform

Project Type: AI Infrastructure / Backend Engineering / Developer Tools
Target Roles: AI Platform Engineer, AI Backend Engineer, ML Infrastructure Engineer, Software Engineer
Development Timeline: 6–8 weeks (part-time)
Primary Stack: Python, FastAPI, React, TypeScript, PostgreSQL, Redis, OpenTelemetry, Docker, GitHub Actions

---

1. Executive Summary

EvalScope is a developer platform for evaluating, monitoring, and improving applications powered by large language models (LLMs).

Modern AI applications are difficult to evaluate using traditional software tests. Their outputs can vary between executions, model updates may introduce regressions, and changes to prompts or retrieval pipelines can improve some responses while degrading others.

EvalScope addresses these challenges by providing a centralized system for managing evaluation datasets, executing experiments, collecting traces, comparing model and prompt versions, and identifying quality regressions.

Developers can use the platform to answer questions such as:

- Did the latest prompt change improve answer quality?
- Which model performs best on our evaluation dataset?
- Did a retrieval pipeline update introduce regressions?
- Why did a particular AI request fail?
- How much latency and cost did a model change introduce?
- Should an updated AI workflow be approved for release?

The system will support both offline evaluation of predefined datasets and online observability of real application requests.

The primary objective is to build a reliable, measurable evaluation infrastructure system rather than another AI chatbot.

2. Problem Statement

LLM applications introduce several engineering challenges.

2.1 Nondeterministic outputs

Unlike traditional functions, LLMs can produce different responses to the same input. Exact-match assertions are often insufficient for measuring quality.

2.2 Lack of standardized evaluation

Developers frequently rely on manual inspection or isolated test scripts. These methods are difficult to reproduce, compare, or automate.

2.3 Limited visibility into failures

A failed AI response may result from an incorrect retrieval result, a poorly designed prompt, model behavior, or a downstream tool failure.

Without structured traces, identifying the root cause is difficult.

2.4 Quality versus performance tradeoffs

A model may provide better answers while introducing higher latency, token usage, or operating costs.

Engineering teams need to compare these dimensions before deploying changes.

2.5 Regression detection

A prompt update that improves average quality may still break important edge cases.

Teams need repeatable evaluations that can detect regressions before release.

3. Project Goals

Functional goals

1. Create and manage versioned evaluation datasets.
2. Execute evaluation experiments across different models, prompts, and retrieval configurations.
3. Support deterministic and LLM-based evaluation metrics.
4. Record structured execution traces and performance metrics.
5. Compare experiment results and identify regressions.
6. Execute evaluations asynchronously with retry and failure recovery.
7. Provide a web dashboard for reviewing results and investigating failures.
8. Integrate evaluation checks into CI/CD workflows.

Engineering goals

- Build a modular backend with clear component boundaries.
- Support concurrent evaluation execution.
- Persist experiment results and execution history.
- Handle provider failures, rate limits, and timeouts.
- Implement reproducible evaluation configurations.
- Measure latency, throughput, and evaluation reliability.
- Establish meaningful automated test coverage.
- Provide reproducible local deployment and benchmark instructions.

Non-goals for the initial release

- Training or fine-tuning foundation models.
- Building a new vector database.
- Implementing a general-purpose workflow orchestration framework.
- Supporting every commercial LLM provider.
- Creating a production-scale SaaS billing system.
- Building a replacement for existing distributed task queue infrastructure.

These features would substantially increase development time without necessarily improving the project's portfolio value.

4. Target Users and Use Cases

User 1: AI application developer

A developer modifies a RAG prompt and wants to verify whether answer quality improved.

Workflow:

1. Select an evaluation dataset.
2. Choose the existing and updated prompt versions.
3. Execute both configurations.
4. Compare quality, latency, and token usage.
5. Inspect cases where the updated prompt performs worse.
6. Decide whether to release the change.

User 2: AI platform engineer

An engineer needs to investigate a slow or unsuccessful AI request.

Workflow:

1. Open the observability dashboard.
2. Find requests with high latency or errors.
3. Inspect the trace timeline.

4. Identify whether the issue occurred during retrieval, generation, or tool execution.
5. Compare the request against historical performance.
6. Document the root cause.

User 3: Engineering team

A team wants to prevent prompt regressions from reaching production.

Workflow:

1. Configure a versioned evaluation dataset.
2. Define quality and latency thresholds.
3. Trigger an evaluation through GitHub Actions.
4. Compare results against a baseline.
5. Fail the quality gate if release criteria are violated.
6. Review the regression report.

5. System Architecture

EvalScope will use a modular architecture with independent responsibilities for experiment management, execution, evaluation, and observability.

5.1 Frontend

Technologies: React, TypeScript, Vite, TanStack Query, Tailwind CSS, Recharts.

Responsibilities:

- Manage datasets and test cases.
- Configure experiments.
- Display evaluation progress.
- Compare experiment results.
- Visualize latency and quality metrics.
- Inspect individual execution traces.
- Review regressions and failed cases.

5.2 API service

Technologies: Python, FastAPI, Pydantic, SQLAlchemy, PostgreSQL.

Responsibilities:

- Expose REST APIs.
- Validate experiment configurations.
- Manage datasets and version history.

- Create evaluation runs.
- Retrieve evaluation results.
- Calculate aggregate statistics.
- Enforce application-level permissions.
- Coordinate asynchronous evaluation execution.

5.3 Evaluation engine

Technologies: Python, provider SDKs, custom evaluator interfaces.

Responsibilities:

- Execute prompts against configured models.
- Run deterministic evaluation functions.
- Execute LLM-as-judge evaluations.
- Evaluate retrieval quality.
- Record evaluator outputs and explanations.
- Handle provider errors and timeouts.

The engine will use a pluggable evaluator interface so that new evaluation methods can be added without modifying the experiment orchestration logic.

5.4 Background execution

Technologies: Redis and a Python worker system such as Celery or Dramatiq.

Responsibilities:

- Schedule evaluation jobs.
- Execute test cases concurrently.
- Apply provider-specific concurrency limits.
- Retry transient failures with exponential backoff.
- Record progress and failures.
- Support cancellation and resumable execution.

EvalScope will use its own small Python worker implementation initially rather than copying the architecture of the existing Go Distributed Task Queue.

The focus here is evaluation infrastructure and observability, not rebuilding durable queue semantics.

5.5 Observability

Technologies: OpenTelemetry, OpenTelemetry Collector, Jaeger, Prometheus, Grafana.

Responsibilities:

- Record distributed traces.
- Track model request latency.
- Record retrieval and tool-execution spans.
- Capture provider errors.
- Measure token usage and estimated cost.
- Provide aggregate service metrics.

PostgreSQL will store evaluation-specific records, while the tracing backend will store detailed execution traces.

5.6 Data storage

PostgreSQL will store:

- Projects
- Dataset versions
- Evaluation test cases
- Prompt versions
- Experiment configurations
- Evaluation runs
- Individual results
- Scores
- Baselines
- Regression reports

Redis will support:

- Job queues
- Short-lived execution state
- Rate-limit coordination
- Temporary caching

6. High-Level Data Flow

1. A developer creates an evaluation dataset.
2. The developer selects a model and prompt configuration.
3. The API creates an immutable evaluation run record.
4. The backend schedules individual test-case executions.
5. Workers invoke the configured LLM or RAG application.
6. OpenTelemetry captures the execution trace.
7. Evaluators calculate quality scores.
8. Results and execution metadata are persisted.
9. The aggregation service computes experiment-level metrics.
10. The dashboard displays results and comparisons.
11. The regression engine evaluates release thresholds.

Each evaluation result must be traceable to the exact dataset version, model configuration, prompt version, and evaluator version used to produce it.

7. Core Functional Modules

Module A: Evaluation Dataset Management

Users can create datasets containing input questions, expected outputs, and optional reference documents.

Example test case:

```
{
  "case_id": "rag-001",
  "input": "What is the refund policy?",
  "reference_answer": "Refunds are available within 30 days.",
  "reference_contexts": [
    "Customers may request refunds within 30 days."
  ],
  "tags": ["policy", "basic"]
}
```

Features:

- JSONL and CSV imports.
- Dataset versioning.
- Test-case tagging.
- Validation of required fields.
- Immutable snapshots for evaluation runs.
- Dataset-level summary statistics.

Acceptance criteria: Re-running an experiment against the same dataset version must use the same test cases, even if the original dataset is subsequently edited.

Module B: Experiment Management

Users configure evaluation experiments using:

- Dataset version.
- Model provider and model identifier.
- Prompt version.
- Temperature and other generation settings.
- Evaluator configuration.
- Concurrency limit.
- Retry policy.
- Baseline experiment.

Example:

```
{
  "dataset_version_id": "dataset-v3",
  "model": "configured-model-id",
  "prompt_version": "prompt-v2",
  "temperature": 0,
  "evaluators": [
    "exact_match",
    "semantic_similarity",
    "llm_judge"
  ],
  "concurrency": 5
}
```

Every run stores a frozen configuration.

Model identifiers and prompt versions are recorded for reproducibility, but external provider behavior may still change over time.

Module C: Evaluation Engine

The initial release will support four evaluation methods.

1. Exact Match

Compares normalized predicted outputs against reference answers.

Useful for structured outputs and deterministic tasks.

2. Semantic Similarity

Compares embeddings of generated answers and reference answers.

Useful as a similarity signal, but not sufficient by itself to establish factual correctness.

3. Retrieval Evaluation

Measures retrieval quality using labeled relevant documents.

Metrics:

- Precision@K
- Recall@K
- Mean Reciprocal Rank (MRR)

4. LLM-as-Judge

Uses a configured model to score generated answers according to a rubric.

Example scoring dimensions:

- Correctness
- Relevance
- Completeness
- Groundedness

The judge returns structured results:

```
{
  "score": 4,
  "max_score": 5,
  "reason": "The response is accurate but omits one condition.",
  "evaluator_version": "judge-v1"
}
```

To avoid presenting LLM judgments as objective truth, EvalScope will also support human-reviewed labels and judge calibration.

Module D: Execution Tracing

Each AI request will produce a trace containing relevant execution stages.

Example:

```
Evaluation Run
  └── Test Case
        ├── Retrieve Documents
        ├── Construct Prompt
        ├── LLM Generation
        ├── Evaluate Response
        └── Persist Results
```

Each span should include:

- Trace ID
- Span ID
- Operation name
- Start and end timestamps
- Duration
- Status
- Error classification
- Model identifier when applicable
- Token counts when available

Sensitive prompt content should not be automatically recorded in tracing attributes. Full inputs and outputs should be stored only when explicitly enabled and governed by appropriate access controls.

Module E: Experiment Comparison

Users can compare two experiment runs executed against the same dataset.

Metrics include:

- Average evaluation score.
- Pass rate.
- p50 and p95 latency.
- Total input and output tokens.
- Estimated API cost.
- Error rate.
- Per-test-case score differences.

The system should distinguish between:

- Improved cases.
- Regressed cases.
- Unchanged cases.
- Cases with missing or failed evaluations.

Comparisons must reject or clearly flag incompatible dataset versions or evaluator configurations.

Module F: Regression Detection

Users define release thresholds.

Example:

```
quality_gate:
  minimum_pass_rate: 0.90
  maximum_pass_rate_drop: 0.03
  maximum_p95_latency_ms: 2500
  maximum_error_rate: 0.02
```

The regression engine compares the candidate experiment with its baseline.

Possible outcomes:

- PASS
- FAIL
- INCONCLUSIVE

An inconclusive result may occur when too many test cases fail to execute or the evaluation data is insufficient.

For statistical rigor, comparisons should use paired test-case results and report uncertainty rather than relying solely on average scores.

Module G: CI/CD Integration

A GitHub Actions workflow will:

1. Start a lightweight evaluation run.
2. Wait for completion.
3. Retrieve aggregate metrics.
4. Compare results against a baseline.
5. Generate a machine-readable report.
6. Return a passing or failing exit code.

The initial integration will use a CLI command:

```
evalscope run \
  --config eval.yaml \
  --baseline main \
  --fail-on-regression
```

The CLI will communicate with the EvalScope API.

For pull requests, the workflow should run a small, deterministic evaluation suite. More expensive LLM evaluations can run separately to avoid excessive cost.

8. Database Design

The initial schema will contain the following principal tables.

Table	Purpose
projects	Organize evaluation applications
datasets	Dataset identity and metadata
dataset_versions	Immutable dataset snapshots
test_cases	Inputs and reference labels
prompt_versions	Versioned prompt templates
experiments	Experiment definitions
evaluation_runs	Individual executions
evaluation_results	Per-case model outputs
evaluator_scores	Scores and explanations
baselines	Approved reference runs

regression_reports Comparison and release decisions

Important relationships:

- One project has many datasets.
- One dataset has many versions.
- One experiment has many evaluation runs.
- One run has many evaluation results.
- One result has multiple evaluator scores.

Data integrity requirements

- Use UUID primary keys.
- Enforce foreign-key relationships.
- Store timestamps in UTC.
- Prevent mutation of completed run configurations.
- Make result persistence idempotent.
- Enforce unique constraints on run/test-case/attempt identities.
- Use database transactions for state transitions.
- Record explicit failure states instead of silently dropping failed evaluations.

9. REST API Design

Method	Endpoint	Purpose
POST	`/api/v1/datasets`	Create dataset
POST	`/api/v1/datasets/{id}/versions`	Create dataset version
GET	`/api/v1/datasets/{id}`	Retrieve dataset
POST	`/api/v1/experiments`	Create experiment
POST	`/api/v1/runs`	Start evaluation run
GET	`/api/v1/runs/{id}`	Retrieve run status
GET	`/api/v1/runs/{id}/results`	Retrieve results
POST	`/api/v1/comparisons`	Compare runs
GET	`/api/v1/traces/{trace_id}`	Retrieve trace metadata
POST	`/api/v1/quality-gates/check`	Evaluate release thresholds

Long-running operations should return HTTP `202 Accepted` and a run identifier rather than holding the HTTP connection open.

Run statuses:

```
PENDING
   ↓
RUNNING
   ├── COMPLETED
```

```
├── PARTIAL
├── FAILED
└── CANCELED
```

Individual test-case execution states will be tracked separately from aggregate run status.

10. Reliability and Performance Design

Asynchronous execution

Evaluation jobs will run outside the request-response cycle.

Workers should support configurable concurrency and backpressure.

Retry strategy

Transient failures such as rate limits and network timeouts will be retried with exponential backoff and jitter.

Permanent errors such as invalid credentials or malformed requests should fail without repeated retries.

Idempotency

Each evaluation execution will have a stable identity.

Retries must not create duplicate logical results.

Provider calls themselves may be repeated after uncertain failures, so the system must not claim exactly-once external inference.

Rate limiting

The platform will enforce provider-level concurrency and request-rate limits.

Cancellation

Users can cancel pending evaluations.

For running requests, cancellation will be best-effort and may not stop a request already accepted by an external provider.

Failure recovery

Workers must persist execution outcomes and support recovery of unfinished tasks after process restarts.

Performance optimization

Potential techniques include:

- Batch database writes.
- Bounded concurrency.
- Connection pooling.
- Efficient pagination.
- Caching repeated deterministic evaluator results.
- Avoiding unnecessary LLM judge calls.

11. Observability and Monitoring

Application metrics

- `evaluation_runs_total`
- `evaluation_cases_total`
- `evaluation_failures_total`
- `evaluation_duration_seconds`
- `llm_request_duration_seconds`
- `llm_tokens_total`
- `evaluation_queue_depth`
- `evaluation_worker_active`
- `evaluation_retries_total`

Metrics must use bounded label values. Prompt content, user IDs, and individual trace IDs must not be used as Prometheus labels.

Dashboards

System Health Dashboard

- Request rate
- Error rate
- API latency
- Queue depth
- Worker utilization

AI Evaluation Dashboard

- Quality scores over time
- Pass rates
- Model comparison

- Latency distribution
- Token usage
- Estimated cost

Failure Investigation Dashboard

- Failed evaluation cases
- Error categories
- Trace timelines
- Retrieval failures
- Model timeouts

12. Security and Data Governance

The initial local prototype will use development credentials and synthetic datasets.

Before exposing the service publicly, the following controls should be implemented:

- Authentication and project-scoped authorization.
- Secure provider API key storage.
- TLS for external connections.
- Input validation and request-size limits.
- Secret redaction in logs and traces.
- Configurable retention of prompts and responses.
- Safe handling of uploaded datasets.
- Protection against prompt injection in retrieved or user-provided content.
- Separation of evaluation data across projects.

The project should not claim HIPAA or SOC 2 compliance without the corresponding controls, processes, and independent validation.

13. Testing Strategy

Unit tests

Test:

- Evaluator calculations.
- Dataset validation.
- Configuration parsing.
- Score aggregation.
- Regression thresholds.
- Cost calculations.

Integration tests

Test:

- Dataset creation and versioning.
- Evaluation run lifecycle.
- Worker-to-database interactions.
- Idempotent result persistence.
- Failed-provider responses.
- API authorization.

Failure-injection tests

Simulate:

- Provider timeouts.
- HTTP 429 rate limiting.
- Worker termination.
- Database disconnections.
- Duplicate task delivery.
- Malformed model responses.
- Partial evaluation completion.

End-to-end tests

Using Playwright:

1. Upload an evaluation dataset.
2. Configure an experiment.
3. Execute the experiment.
4. Inspect results.
5. Compare two runs.
6. View regression findings.

Deterministic testing

Most automated tests should use mocked or recorded model responses.

Live provider tests should run separately with explicit budgets and clear reporting of nondeterministic outcomes.

14. Benchmarking Plan

The project must provide reproducible measurements rather than unsupported performance claims.

Experiment A: Evaluation throughput

Run datasets containing:

- 100 test cases.
- 1,000 test cases.
- 5,000 test cases.

Measure completion throughput under different worker counts.

Use a simulated provider with controlled latency to isolate orchestration overhead.

Experiment B: Concurrency scaling

Compare:

- 1 worker.
- 3 workers.
- 5 workers.

Measure throughput, queue waiting time, and error rates.

Experiment C: Failure recovery

Inject transient provider failures and worker crashes.

Measure:

- Recovery success rate.
- Duplicate logical result count.
- Recovery time.
- Retry behavior.

Experiment D: Evaluation quality

Create a manually labeled dataset of at least 100 examples, including correct answers, hallucinations, incomplete responses, and irrelevant responses.

Compare LLM-as-judge scores with human labels.

Measure agreement and investigate disagreements.

Experiment E: Regression detection

Introduce controlled prompt regressions.

Measure whether the quality gate identifies the affected cases and whether the release decision is consistent with predefined thresholds.

Performance targets

These are proposed acceptance targets, not existing results:

Metric	Initial target
Duplicate persisted logical results	0 in tested failure scenarios
Completed result persistence	100% in successful test runs
Recovery after worker termination	Within 60 seconds in local tests
Evaluation dataset	At least 100 human-labeled cases
Experiment comparison	Per-case and aggregate results
Test suite	Unit, integration, and E2E coverage
Reproducibility	Documented setup and benchmark commands

Actual throughput and latency targets should be established after measuring a baseline on documented hardware.

15. Development Roadmap

Week 1 — Architecture and Foundation

- Initialize repository and development environment.
- Create Docker Compose configuration.
- Implement FastAPI application structure.
- Set up PostgreSQL schema and migrations.
- Define dataset and experiment models.
- Configure testing and GitHub Actions.

Deliverable: Working backend foundation and database schema.

Week 2 — Dataset and Experiment APIs

- Implement dataset CRUD.
- Add immutable dataset versions.
- Implement experiment configuration.
- Add validation and error handling.
- Build initial API tests.

Deliverable: Developers can create datasets and configure experiments.

Week 3 — Evaluation Execution Engine

- Implement provider abstraction.
- Add asynchronous workers.

- Implement exact-match and retrieval metrics.
- Add retry and timeout handling.
- Persist results and execution states.

Deliverable: Complete offline evaluation pipeline.

Week 4 — Advanced Evaluation

- Add semantic similarity.
- Add LLM-as-judge evaluation.
- Implement evaluator versioning.
- Create a labeled evaluation dataset.
- Implement aggregate scoring.

Deliverable: Multiple evaluation methods with reproducible results.

Week 5 — Observability

- Integrate OpenTelemetry.
- Configure Jaeger, Prometheus, and Grafana.
- Instrument provider and evaluator execution.
- Record latency and token usage.
- Build trace inspection APIs.

Deliverable: Traceable evaluation executions and system metrics.

Week 6 — Frontend Dashboard

- Implement experiment dashboard.
- Build dataset management views.
- Display evaluation results.
- Add model/prompt comparisons.
- Add per-case failure investigation.

Deliverable: End-to-end usable web application.

Week 7 — Regression Detection and CI

- Implement baseline comparison.
- Add quality-gate configuration.
- Build CLI.
- Integrate GitHub Actions.
- Add failure-injection tests.

Deliverable: Automated AI regression checks.

Week 8 — Benchmarking and Polish

- Run load tests.
- Record benchmark results.
- Improve error handling.
- Document architecture and tradeoffs.
- Capture dashboard screenshots.
- Record a short demonstration.
- Finalize README and resume metrics.

Deliverable: Portfolio-ready project with reproducible evidence.

16. Minimum Viable Product

If time is limited, the MVP should include:

1. Versioned evaluation datasets.
2. Asynchronous evaluation runs.
3. Exact-match and LLM-as-judge evaluation.
4. PostgreSQL result persistence.
5. Experiment comparison.
6. Basic OpenTelemetry tracing.
7. A React dashboard.
8. Automated integration tests.

The MVP should be usable without implementing every planned feature.

The highest-value additions after the MVP are retrieval evaluation, regression detection, and CI integration.

17. Repository Structure

```
evalscope/
├── backend/
│   ├── app/
│   │   ├── api/
│   │   ├── core/
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── services/
│   │   ├── evaluators/
│   │   ├── providers/
│   │   ├── workers/
│   │   └── telemetry/
│   ├── migrations/
│   └── tests/
├── frontend/
│   ├── src/
```

```
│   │   ├── components/
│   │   ├── pages/
│   │   ├── features/
│   │   ├── hooks/
│   │   └── services/
│   └── tests/
├── cli/
├── datasets/
│   └── examples/
├── benchmarks/
├── deploy/
│   ├── docker/
│   ├── prometheus/
│   └── grafana/
├── docs/
│   ├── architecture.md
│   ├── database.md
│   ├── evaluation.md
│   ├── reliability.md
│   └── benchmarks.md
├── .github/
│   └── workflows/
├── docker-compose.yml
├── Makefile
└── README.md
```

18. Documentation Requirements

The repository should include:

README.md

- Problem statement.
- Main features.
- Architecture overview.
- Quick start.
- Demo screenshots.
- Example evaluation workflow.
- Testing instructions.
- Benchmark summary.
- Known limitations.

Architecture document

- Component responsibilities.
- Data flow.
- Execution lifecycle.
- Design decisions and tradeoffs.

Evaluation methodology

- Metric definitions.
- Human labeling procedure.
- Judge rubric.
- Evaluation limitations.
- Comparison methodology.

Reliability document

- Retry behavior.
- Failure handling.
- Idempotency boundaries.
- Worker recovery.
- Known failure scenarios.

Benchmark report

- Hardware and environment.
- Dataset sizes.
- Experimental procedure.
- Raw measurements.
- Results and limitations.

19. Expected Portfolio Outcomes

By completing EvalScope, the project should demonstrate:

AI engineering

- LLM evaluation.
- Prompt and model experimentation.
- RAG quality measurement.
- LLM-as-judge design.
- AI regression testing.

Backend engineering

- REST API design.
- Asynchronous execution.
- Database modeling.
- Retry and failure recovery.
- Concurrency control.

Infrastructure

- Distributed tracing.
- Metrics and monitoring.

- Containerization.
- CI/CD integration.
- Reproducible benchmarking.

Full-stack engineering

- React/TypeScript dashboards.
- API integration.
- Data visualization.
- Interactive debugging workflows.

20. Proposed Resume Description

The following bullets are templates for use **after implementation and validation**, not claims about completed work.

EvalScope — LLM Evaluation & Observability Platform
Python, FastAPI, React, TypeScript, PostgreSQL, Redis, OpenTelemetry, Docker

- Built an AI evaluation platform supporting versioned datasets, asynchronous experiment execution, and automated quality assessment across LLM and RAG workflows.
- Implemented evaluation pipelines combining deterministic metrics, retrieval-quality scoring, and LLM-as-judge assessments, with experiment comparison and regression detection.
- Integrated OpenTelemetry tracing and performance monitoring to analyze model latency, token usage, errors, and evaluation outcomes; developed CI quality gates to detect regressions before release.

After benchmarking, replace a general statement with one specific, verified performance or reliability result.

21. Definition of Done

The project will be considered portfolio-ready when:

- A new developer can launch the application using documented commands.
- A dataset can be uploaded and versioned.
- An experiment can be configured and executed.
- Individual results and evaluator scores are persisted.
- Two compatible experiments can be compared.
- A failed evaluation can be investigated through a trace.
- A regression can cause a CI quality gate to fail.
- Unit and integration tests pass.
- Failure recovery has been exercised.
- Benchmark results are reproducible.

- The README clearly explains architectural tradeoffs and limitations.

Final outcome: A functioning developer platform that demonstrates how to evaluate, observe, and improve LLM applications systematically, with measurable engineering evidence suitable for AI platform and backend software engineering interviews.